# Graficación por Computadora
# Tarea 6

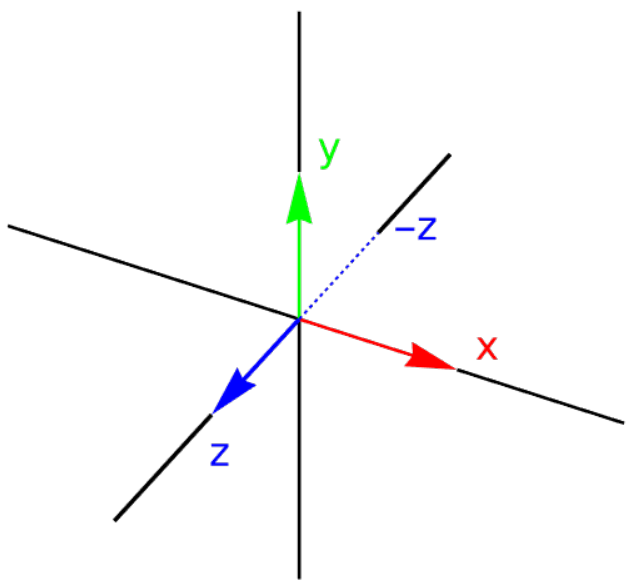
Francisco Alejandro Arganis Ramírez

30 de abril de 2026

## 1 Descripción

### 1.1 Sistema coordenado

Siguiendo las convenciones de OpenGL, se usa un sistema coordenado de mano derecha tal que la dirección "hacia enfrente" coincide con el eje $-Z$, la dirección "hacia la derecha" coincide con el eje $X$ y la dirección "hacia arriba" coincide con el eje $Y$.




Esto en contraste con el sistema coordenado usado en entregas anteriores donde las direcciones

hacia enfrente, derecha y arriba coinciden con los ejes $Y$, $X$ y $Z$, respectivamente.

## 1.2 Inicialización

Al iniciar la ejecución, el programa carga los modelos en los archivos `obj/bunny.obj`, `obj/dragon.obj`, `obj/Happy_Buddha.obj`, `obj/Teapot.obj`, `obj/Cube_Quads`, `obj/Sphere_Patches.obj` y `obj/Nathan.obj` usando la biblioteca `tinyobjloader`. Si el modelo no contiene información de normales para cada vértice, éstas se calculan manualmente por triángulo. Si el modelo no contiene información de coordenadas de textura para cada vértice, se asignan valores predeterminados de coordenadas de textura.

También se cargan las imágenes `textures/Madera.png`, `textures/Ladrillos.png`, `textures/Baldosas.png`, `textures/Placas.png` y `textures/Nathan.png`.

Después de leer los modelos e imágenes, se inicializa una ventana con Qt. La estructura `Canvas` hereda de `QOpenGLWidget`, lo cual permite tener un widget que muestre una imagen renderizada con OpenGL.

En el método `initializeGL` se realiza los siguiente:

1. Se activan las opciones para realizar la prueba de profundidad, oclusión de caras traseras, mezcla de colores, normalización de normales y cálculos de iluminación.
2. Se asignan los parámetros de la función de mezcla de colores.
3. Se asignan los valores para el componente ambiental global de la escena.
4. Se preparan las texturas para su uso en el renderizado. Se crean identificadores para las mismas, se especifican los datos de las imágenes y su formato y finalmente se construyen los mipmaps.
5. Se asigna la función de textura `GL_MODULATE` para que, en cada fragmento, se multiplique el valor muestreado de la imagen por el resultados de iluminación.
6. Se asigna el modo de generación de texturas `GL_NORMAL_MAP`.

## 1.3 Escena e interfaz de usuario

La escena consiste en 7 objetos, 2 fuentes de luz puntuales y 3 cámaras. Solo 1 objeto es visible en un momento dado.

La interfaz gráfica de usuario permite selecionar la cámara actual desde la cual se renderiza la escena, así como modificar su posición y rotación y elegir entre proyección ortogonal o en prespectiva. Se proporciona un botón para reorientar la cámara seleccionada hacia la posición actual del objeto visible.

La interfaz también permite seleccionar el objeto visible. Inicialmente el objeto visible está en el origen y se puede modificar su posición, rotación y escalamiento mediante los controles de la interfaz.

Una luz es blanca con componentes ambiental $(0.1, 0.1, 0.1, 1)$, difuso $(1, 1, 1, 1)$ y especular $(1, 1, 1, 1)$. La otra luz es azul, con componentes ambiental $(0.04, 0.16, 0.2, 1)$, difuso $(0.2, 0.8, 1, 1)$ y especular

$(0.2, 0.8, 1, 1)$. Los controles en la interfaz permiten apagar o encender luces y modificar los valores de su posición y componentes.

En la interfaz se puede seleccionar un material para sombrear el objeto visible. El primer material tiene coeficientes ambiental $(0, 0, 0, 1)$, difuso $(0.5, 0.5, 0.5, 1)$, especular $(0.7, 0.7, 0.7, 1)$ y brillo 32. El segundo material tiene coeficientes ambiental $(0.23125, 0.23125, 0.23125, 1)$, difuso $(0.2775, 0.2775, 0.2775, 1)$, especular $(0.773911, 0.773911, 0.773911, 1)$ y brillo 89.6. Con los controles de la interfaz se puede modificar los coeficientes de los materiales.

La interfaz también permite seleccionar una textura para realizar mapeo de texturas y activar o desactivar la opcción de generar coordenadas de textura automáticamente.

## 1.4 Renderizado

Para renderizar la escena, en el método `paintGL` de la estructura `Canvas` se realiza lo siguiente:

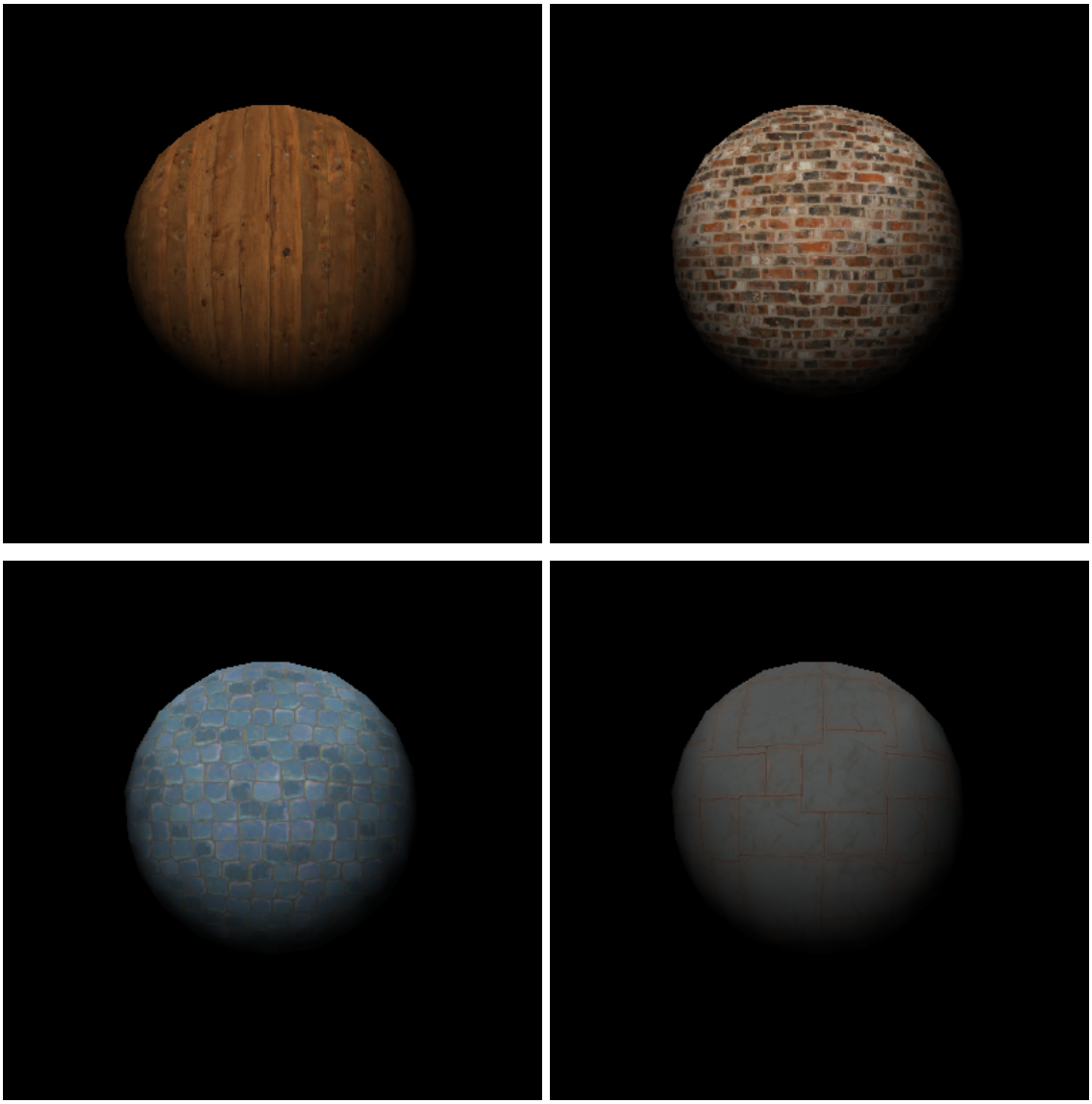
1. Se limpian los buffers de color y profundidad usando `glClear`.
2. Se construye la matriz de vista en la pila `GL_MODELVIEW`.
3. Se asignan las propiedades de las luces.
4. Se construye el producto de la matriz de vista por la matriz de modelo en la pila `GL_MODELVIEW`.
5. Se construye la matriz de proyección en la pila `GL_PROJECTION`.
6. Se asignan los valores del material.
7. Opcionalmente, se activan la apliación de texturas y se liga la textura correspondiente.
8. Opcionalmente, se activa la generación de coordenadas de textura.
9. Se usan `glBegin` y `glEnd` para dibujar los triángulos del modelo del objeto visible. Entre ambas llamdas, se usa un ciclo para iterar los vértices y asignar su normal, coordenadas de textura y posición con `glNormal3dv`, `glTexCoord2dv` y `glVertex3dv`, respectivamente.
10. Se intercambian los buffers.

## 1.5 Mapeo de texturas

Ninguno de los modelos proporcionados tiene vértices con coordenadas de textura. Los valores predeterminados asignados a las coordenadas de textura faltantes permiten mapear una imagen completa a cada cara del cubo, haciendo que sea el único de estos modelos en los que se puede apreciar directamente el mapeo de texturas.

Por este motivo se incluye un modelo extra, un personaje llamdo Nathan descargado de `https://free3d.com/3d-model/nathan-animated-003-walking-644277.html`, que sí tiene coordenadas de textura en sus vértices.

Adicionalmente, el programa ofrece la opción de usar la funcionalidad de OpenGL de generar automáticamente coordenadas de textura. En este caso, la generación de coordenadas en modo `GL_NORMAL_MAP` permite que se puedan observar las texturas mapeadas a la superficie de la esfera.



## 1.6 Código fuente

La carpeta `src` contiene los archivos con el código fuente:

- `app.cpp`. Define las variables usadas por la aplicación y funciones para cargar modelos y texturas desde archivos.
- `canvas.cpp`. Contiene la implementación de la estructura `Canvas` que hereda de `QOpenGLWidget`, incluyendo los métodos antes mencionados `initializeGL` y `paintGL`.
- `gui.cpp`. Contiene la implementación del resto de las estructuras que heredan de `QWidget`,

usadas para mostrar la ventana y los controles de la interfaz.

- `main.cpp`. Inicializa la ventana e inicia la ejecución de la aplicación.

La carpeta `include` contiene los archivos de cabecera que se incluyen en el código fuente, la carpeta `textures` contiene las imágenes usadas para el mapeo de texturas y la carpeta `obj` contiene los modelos de los objetos.

## 2 Compilar y ejecutar

Para compilar el programa, en Linux, abrir una terminal en la dirección donde se encuentran `src`, `include` y `tiny_obj_loader.h` y ejecutar

```
g++ src/*.cpp -I . -I include $(pkg-config --cflags Qt6OpenGLWidgets
    glu) $(pkg-config --libs Qt6OpenGLWidgets glu) -O3
```

Para iniciar el programa ya compilado, con las carpetas `obj` y `textures` en la misma dirección que el ejecutable `a.out`, ejecutar

```
./a.out
```

Se requiere:

- `gcc` 15.2.0 o superior
- `qt` 6.9.2 o superior
- `pkg-config` 1.8.1 o superior
- `OpenGL` 4.6
- `GLU` 1.3

## 3 Repositorio

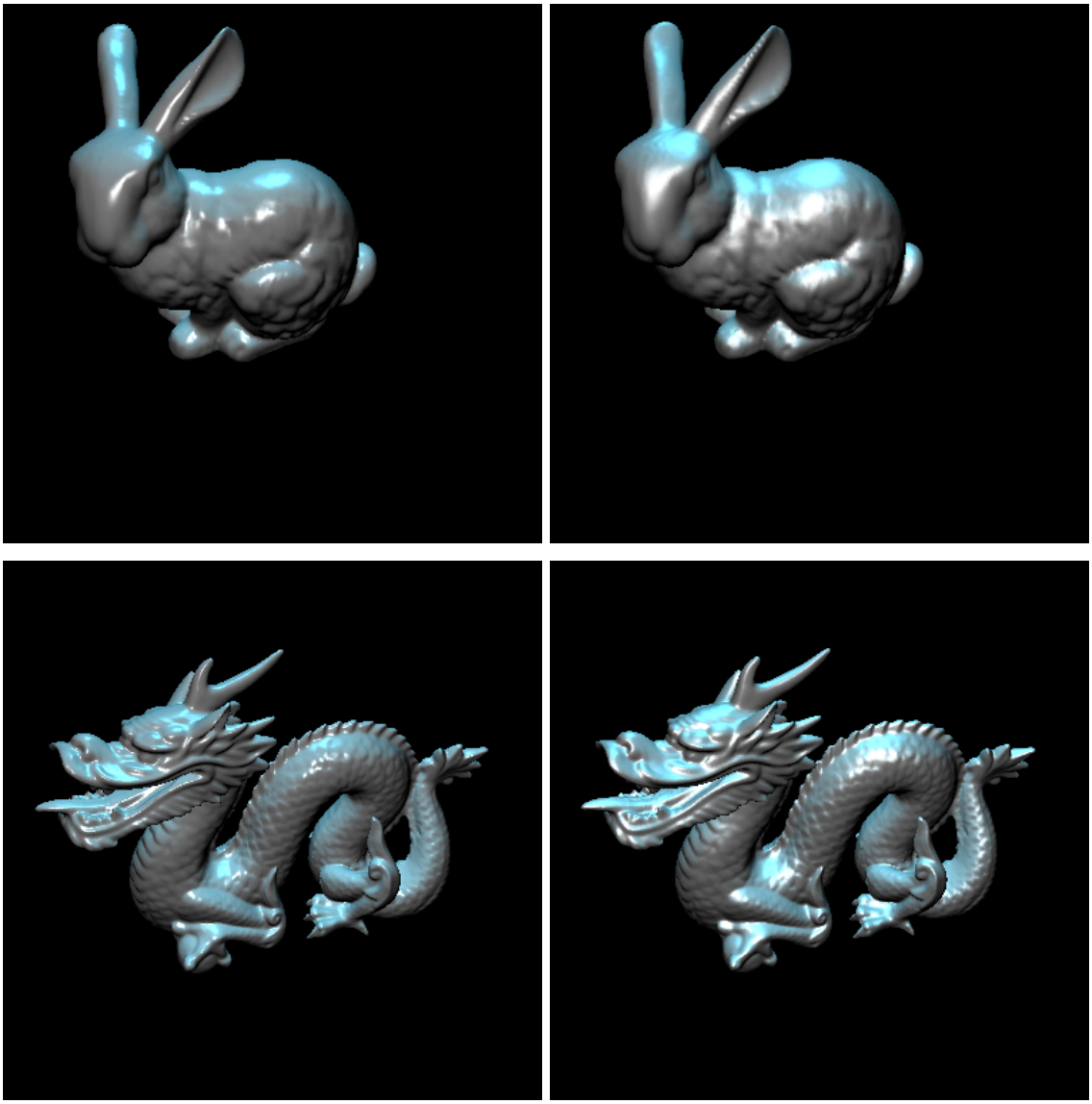
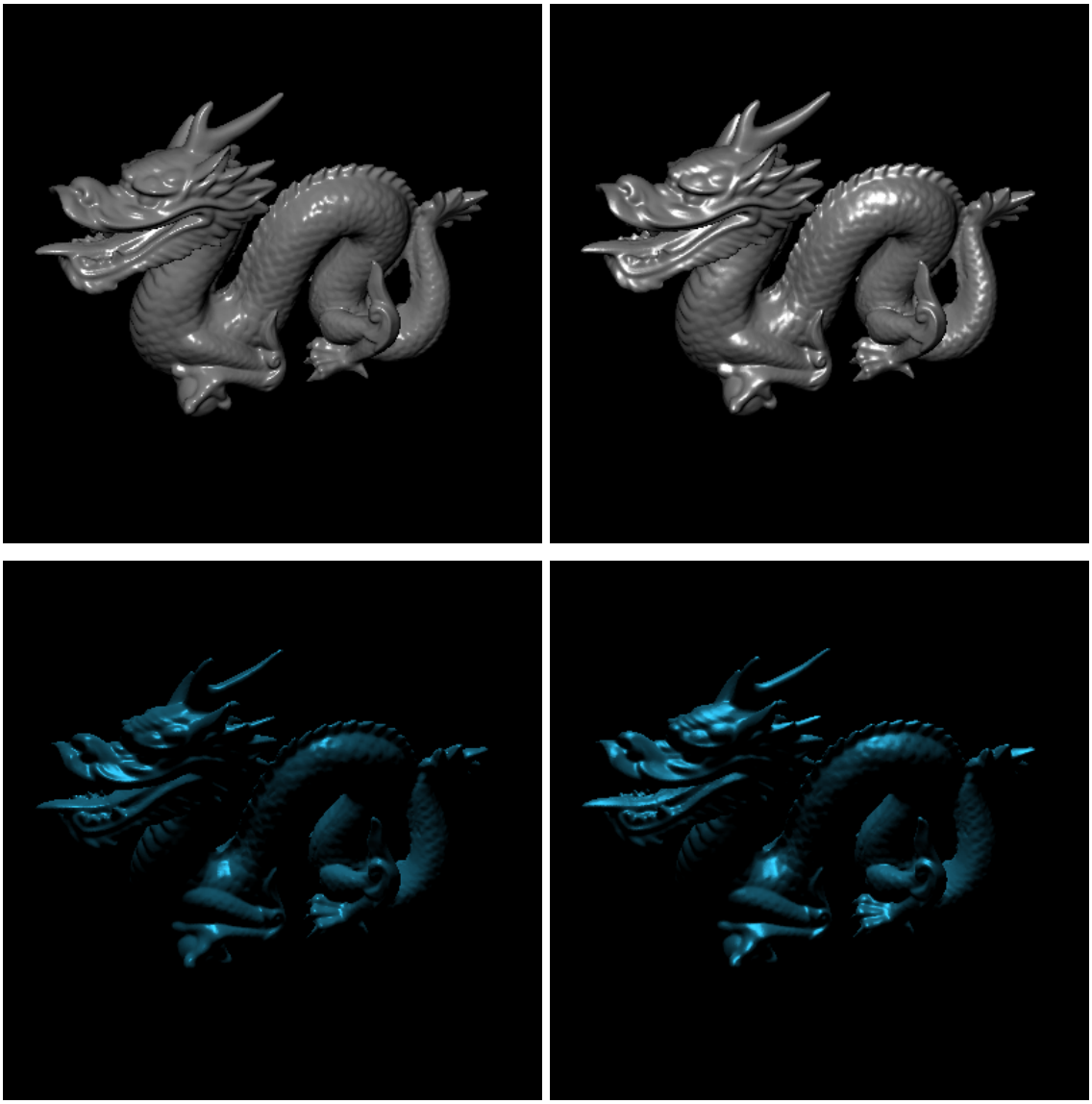
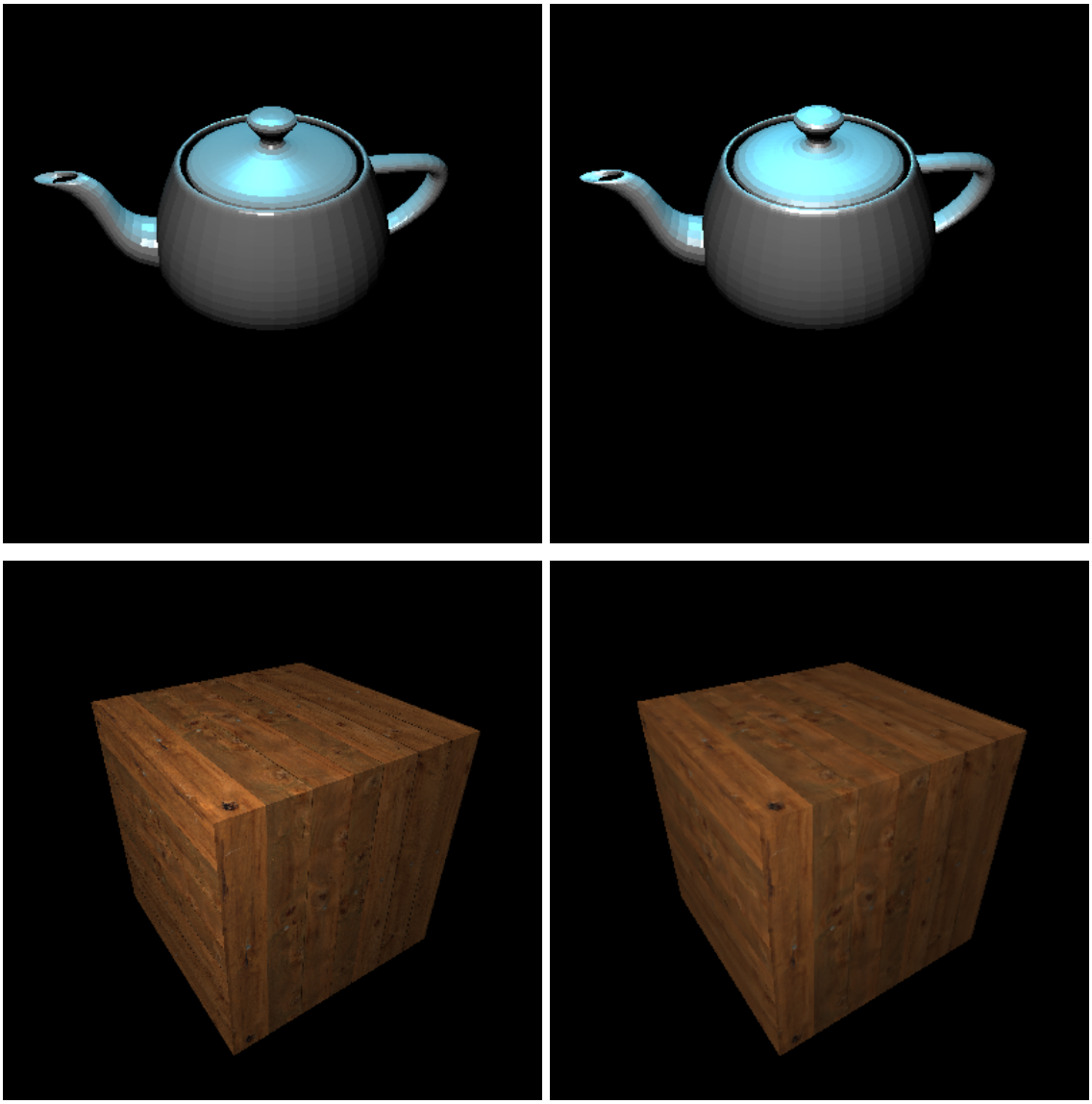
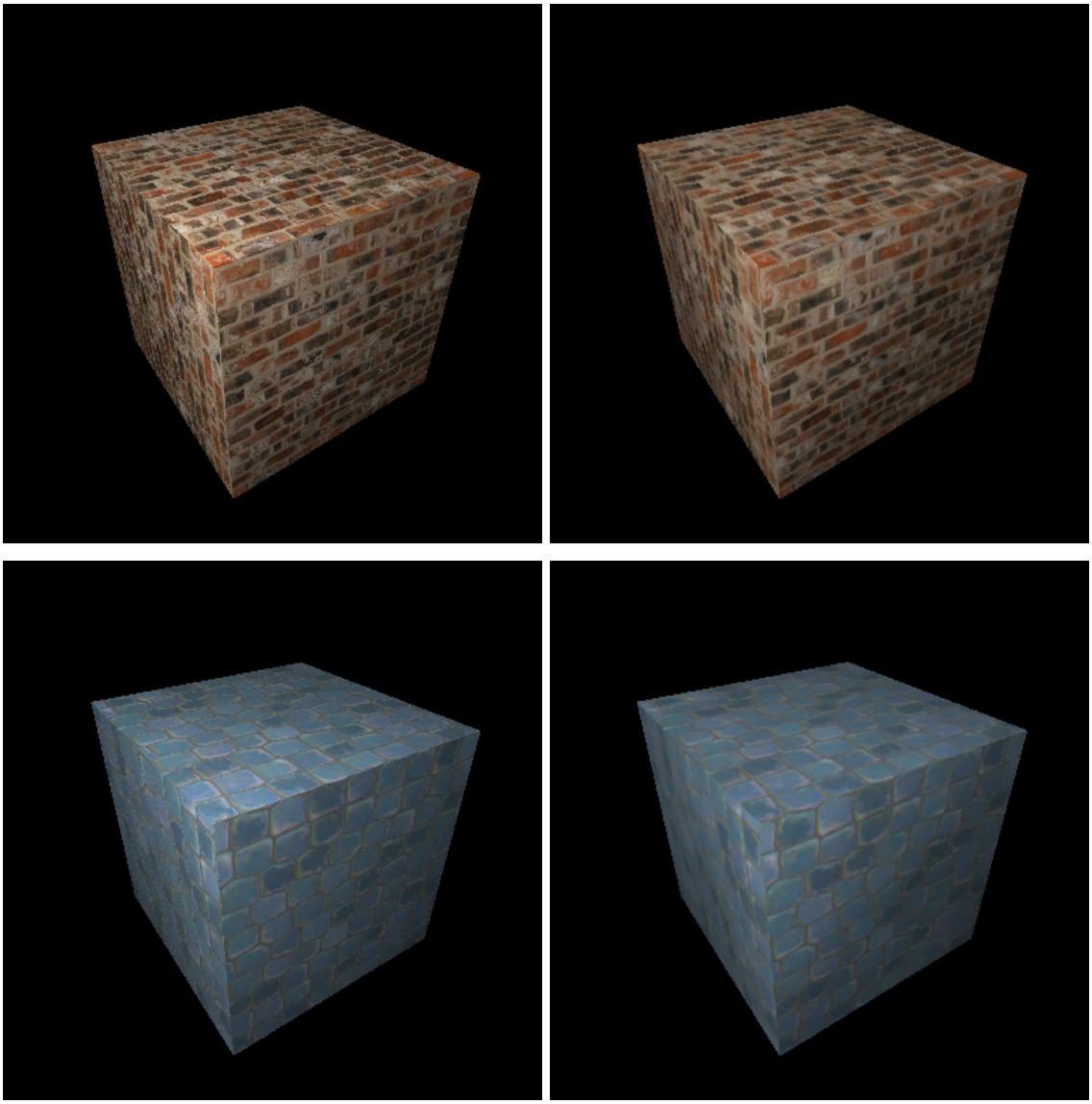
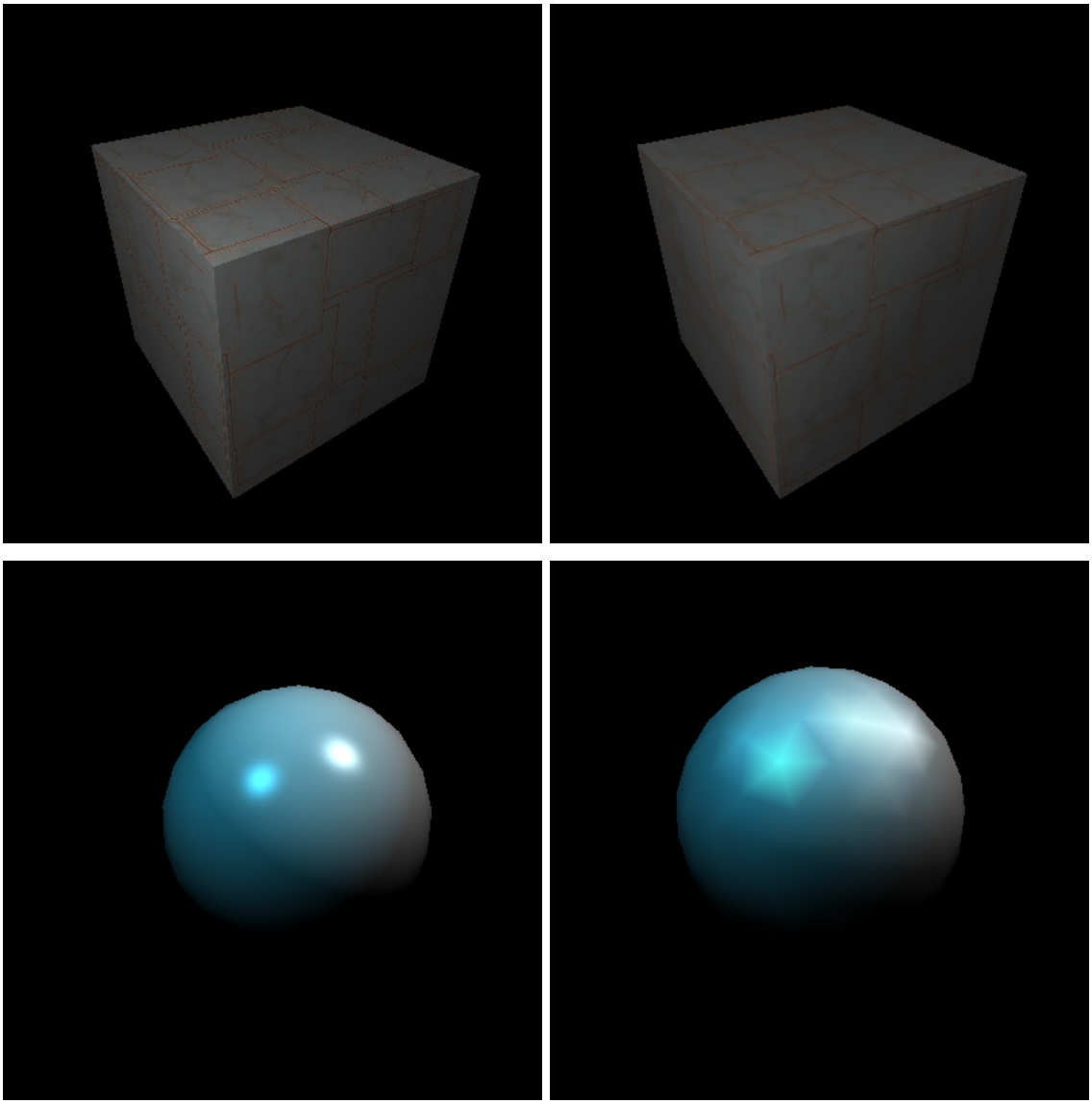
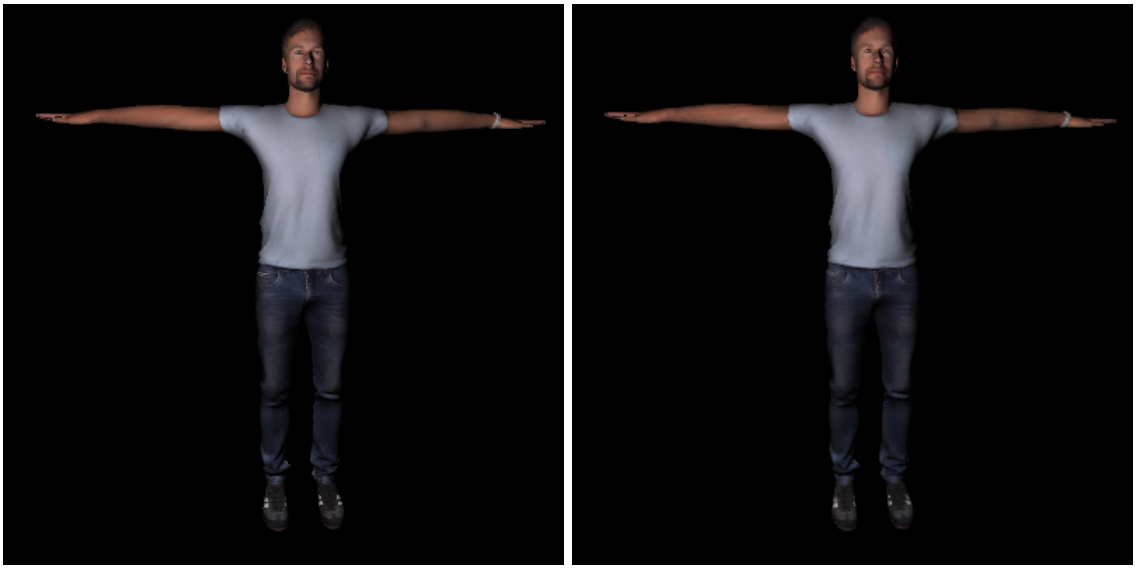
`https://github.com/FranciscoAlejandroArganis/Graficacion_por_Computadora_2026-2`

## 4 Capturas de pantalla

Se incluyen capturas de pantalla del programa en ejecución en la carpeta `screenshots`.

## 5 Imágenes comparativas

A continuación se muestra una secuencia de imágenes generadas con la aplicación del proyecto 1 y la aplicación de la tarea 6. Se pueden observar las diferencias entre los resutlados del sistema de renderizado implementado en entregas anteriores y OpenGL.

Izquierda: proyecto 1.

Derecha: tarea 6 (OpenGL).